

Exámenes-EC-Resueltos.pdf



Anónimo



Estructura de Computadores



2º Grado en Ingeniería Informática



**Facultad de Informática
Universidad Complutense de Madrid**



ESTRUCTURA DE COMPUTADORES

Examen Final - Junio de 2018

Nombre _____ DNI _____
Apellidos _____ Grupo _____

1.- (4 puntos) Sea un procesador segmentado de cinco etapas con las siguientes características:

- Un dato se puede leer y escribir en el banco de registros en el mismo ciclo de reloj.
- Existe anticipación de operandos. El cortocircuito se realiza siempre desde los registros del pipeline, y existe cortocircuito a la etapa ID para las instrucciones de salto, que se resuelven en dicha etapa.
- La detección de riesgos LDE y generación de paradas se realiza en la etapa de decodificación.
- Los riesgos estructurales referidos a memoria se detectan y se resuelven mediante espera en la última etapa de cada unidad funcional.
- Los riesgos de EDE entre dos instrucciones A y B tal que A precede a B se resuelven mediante inhibición de escritura de la instrucción A.
- No se implementan saltos retardados, sino que se busca siempre la siguiente instrucción al salto y se cancela su ejecución si el salto es tomado.
- Las unidades funcionales de las que dispone el procesador son las siguientes:

UF	Cantidad	Latencia	Segmentación
FP ADD	1	3	Sí
FP MUL	1	5	Sí
Int ALU	1	1	No

El siguiente código calcula los elementos del vector `double C[128]` a partir de los elementos de los vectores `double A[128]` y `double B[128]` (recuerda que un elemento en punto flotante de doble precisión ocupa 8 bytes). Los vectores se encuentran almacenados en memoria de forma consecutiva (A-B-C) y la dirección de comienzo del vector A es `0x80001000`.

LOOP:

```
LD      F2, 0(R6)
AND     R2, R4, R9
BEQ     R2, R0, ELSE
IF:     ADDD  F6, F2, F2
        MULD  F6, F8, F6
        BEQ   R0, R0, END
ELSE:   LD    F4, 1024(R6)
        MULD  F6, F2, F4
        ADD   R10, R10, R2
        ADDD  F10, F2, F4
END:    SD    F6, 2048(R6)
ADD     R6, R6, R3
ADD     R4, R4, R9
BNE     R4, R5, LOOP
SUB     R4, R4, R4
```

R6=0x80001000

R3=8

R5=128

R4=0

R9=1

R10=0

R7=-1

F8=5.5

R0 =0 siempre en la
arquitectura MIPS

El contenido inicial de los registros es el siguiente:

- (2 puntos) Representa el diagrama instrucción-tiempo para la primera y segunda iteración e indica los cortocircuitos realizados. Indica claramente las paradas y sus causas.
- (1 punto) Calcula el CPI de la ejecución completa del código.
- (1 punto) Si el procesador utilizara saltos retardados, ¿qué instrucciones llevarías al *delay-slot* de cada uno de los saltos y qué efectos tendría?

2.- (3 puntos) En un sistema con procesador ARM y una memoria de 64KB se quiere analizar el comportamiento del siguiente fragmento de un programa.

WUOLAH

```

.global start
.equ N, 12

.data
A: .word N valores separados por comas
B: .word N valores separados por comas

.bss
C: .space 4*N

.text
....
mov R3, #N
mov R4, #0
mov R5, #0
mov R2, #0
for: cmp R2, R3
    bge fin
    ldr R4, [R0, R2, LSL#2]
    ldr R5, [R1, R2, LSL#2]
    add R4, R4, R5
    str R4, [R7, R2, LSL#2]
    add R2, R2, #1
    b for

```

El programa se enlaza ubicando la sección .data a partir de la **dirección 0x0090** y la sección .bss a continuación de la sección .data, y que el fragmento de la sección .text en la que se encuentra la primera instrucción (mov R3, #N) se ubica en la **dirección 0x0140**

R0, R1 y R7 contienen las direcciones de A, B y C respectivamente

Responde de forma razonada a las siguientes preguntas:

- (0.5 puntos) Obtener el rango de direcciones que ocupa el fragmento de programa y cada uno de los arrays. ¿Cuántos bloques de memoria principal ocupan? ¿Qué bloques son?
- (1.5 puntos) El sistema dispone de caches separadas de datos (D\$) e instrucciones (I\$) de 32 bytes cada una, ambas con política de emplazamiento directo, y políticas de escritura directa (write through) con asignación en escritura (write allocate), y tamaño de bloque de 16 bytes. Obtener los fallos que se producirán en la cache de datos (D\$) y en la cache de instrucciones (I\$) al ejecutar el fragmento del programa. Indicar, además del número total de fallos, para qué bloques de memoria principal se producen los fallos.
- (1 punto) Ahora añadimos al sistema de memoria una cache de segundo nivel unificada para datos e instrucciones (L2\$) de 128 bytes, asociativa por conjuntos de 2 vías, con reemplazamiento LRU y políticas de escritura directa (write through) con asignación en escritura (write allocate). Indicar el número de fallos que se producen en L2 tras ejecutar el fragmento del programa. (El tamaño de bloque es también de 16 bytes).

3.- (3 puntos) En una empresa dedicada al análisis de datos tienen que utilizar un algoritmo que requiere sumar todos los elementos de cada columna de una matriz de 512x512 elementos. El código C de dicho algoritmo es el siguiente:

```

for (j = 0; j < 512; j++)
    for (i = 0; i < 512; i++)
        red[j] = red[j] + A[i][j];

```

Sabiendo que:

- En C los elementos de una matriz se emplazan en memoria consecutivos por filas
- El procesador es de 32 bits (direcciones virtuales de 32 bits).
- El array de enteros A[512][512] se ubica en el mapa virtual de memoria del proceso a partir de la dirección **0x00458000**, y el array de enteros red[512] se ubica justo a continuación de A.
- El sistema operativo (SO) maneja páginas de 4KB con reemplazo **LRU**.

se pide:

- (0.75 puntos) Indicar qué páginas de memoria virtual ocupan los arrays A y red, indicando qué elementos de cada array están en cada página.
- (1.25 puntos) El SO le asigna al proceso 8 marcos de página (lo que sería equivalente a decir que, desde el punto de vista de este proceso, la memoria física tiene 8 páginas). Sabiendo que 2 de esos 8 marcos están ocupados por la página de código y la página de pila, accedidos constantemente y que por tanto nunca son elegidos para el reemplazo, y que los 6 restantes no contienen inicialmente ninguna página de A ni de red; indicar razonadamente cuántos fallos de página se producirán al ejecutar el código y el número de accesos a A y red que no suponen fallo de página.
- (1 punto) Razone si es posible o no reducir significativamente los fallos de página del proceso con alguna modificación en su código, indicando en su caso qué transformación se haría y cuantificando la mejora esperada.



ESTRUCTURA DE COMPUTADORES

Examen Final - Junio de 2018

Nombre _____ DNI _____
Apellidos _____ Grupo _____

1.- (4 puntos) Sea un procesador segmentado de cinco etapas con las siguientes características:

- Un dato se puede leer y escribir en el banco de registros en el mismo ciclo de reloj.
- Existe anticipación de operandos. El cortocircuito se realiza siempre desde los registros del pipeline, y existe cortocircuito a la etapa ID para las instrucciones de salto, que se resuelven en dicha etapa.
- La detección de riesgos LDE y generación de paradas se realiza en la etapa de decodificación.
- Los riesgos estructurales referidos a memoria se detectan y se resuelven mediante espera en la última etapa de cada unidad funcional.
- Los riesgos de EDE entre dos instrucciones A y B tal que A precede a B se resuelven mediante inhibición de escritura de la instrucción A.
- No se implementan saltos retardados, sino que se busca siempre la siguiente instrucción al salto y se cancela su ejecución si el salto es tomado.
- Las unidades funcionales de las que dispone el procesador son las siguientes:

UF	Cantidad	Latencia	Segmentación
FP ADD	1	3	Sí
FP MUL	1	5	Sí
Int ALU	1	1	No

El siguiente código calcula los elementos del vector `double C[128]` a partir de los elementos de los vectores `double A[128]` y `double B[128]` (recuerda que un elemento en punto flotante de doble precisión ocupa 8 bytes). Los vectores se encuentran almacenados en memoria de forma consecutiva (A-B-C) y la dirección de comienzo del vector A es `0x80001000`.

LOOP:

```
LD    F2, 0(R6)
AND   R2, R4, R9
BEQ   R2, R0, ELSE
IF:
    ADDD F6, F2, F2
    MULD F6, F8, F6
    BEQ  R0, R0, END
ELSE:
    LD    F4, 1024(R6)
    MULD F6, F2, F4
    ADD   R10, R10, R2
    ADDD F10, F2, F4
END:
SD     F6, 2048(R6)
ADD    R6, R6, R3
ADD    R4, R4, R9
BNE    R4, R5, LOOP
SUB    R4, R4, R4
```

El contenido inicial de los registros es el siguiente:

R6=0x80001000

R3=8

R5=128

R4=0

R9=1

R10=0

R7=-1

F8=5.5

El registro R0 siempre contiene un 0 en la arquitectura MIPS

- (2 puntos) Representa el diagrama instrucción-tiempo para la primera y segunda iteración e indica los cortocircuitos realizados. Indica claramente las paradas y sus causas.
- (1 punto) Calcula el CPI de la ejecución completa del código.
- (1 punto) Si el procesador utilizara saltos retardados, ¿qué instrucciones llevarías al *delay-slot* de cada uno de los saltos y qué efectos tendría?

Solución

a)

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	
LD	F	D	E	M	W																																					
AND		F	D	E	M	W																																				
BEQ			F	Dp	D	E	M	W																																		
ADDD				Fp	F	X	X	X	X																																	
LD					F	D	E	M	W																																	
MULD						F	Dp	D	T1	T2	T3	T4	T5	M	W																											
ADD						Fp	F	D	E	M	W																															
ADDD							F	D	A1	A2	A3p	A3	M	W																												
SD							Dp	Dp	D	Ep	E	M	W																													
ADD							Fp	Fp	F	Dp	D	E	M	W																												
ADD															Fp	F	D	E	M	W																						
BNE																	F	Dp	D	E	M	W																				
SUB																	Fp	F	X	X	X	X																				
LD																			F	D	E	M	W																			
AND																				F	D	E	M	W																		
BEQ																					F	Dp	D	E	M	W																
ADDD																						Fp	F	D	A1	A2	A3	M	W													
MULD																						F	Dp	Dp	D	T1	T2	T3	T4	T5	M	W										
BEQ																						Fp	Fp	F	D	E	M	W														
LD																																										
SD																																										
ADD																																										
ADD																																										
BNE																																										
SUB																																										

b)

1ª iteración: 19

3ª iteración: Igual que la 1ª

5ª iteración: Igual que la 3ª

...

2ª iteración: 24 al 41 = 18

4ª iteración: Igual que la 2ª

...

Ciclos: $19 \cdot 64 + 18 \cdot 64 + 4 = 2372$

Instrucciones: $11 \cdot 64 + 10 \cdot 64 = 1344$

$CPI = 2372 / 1344 = 1.76$

c)

El ADD de R4 al hueco del primer BEQ. Aprovechamos el hueco y además evitamos la parada del salto BNE al alejar la dependencia.

BEQ R2, R0, ELSE

ADD R4, R4, R9

El primer MULD al hueco del segundo BEQ. Aprovechamos el hueco. Se aleja el MULD del ADD pero, en cambio, se acerca el MULD al SD.

```
BEQ    R0, R0, END
MULD   F6, F8, F6
```

El ADD de R6 al hueco del salto BNE. Aprovechamos el hueco. Se acerca el ADD al LD del comienzo del LOOP, pero la dependencia se resuelve por forwarding sin generar parada.

```
BNE    R4, R5, LOOP
ADD     R6, R6, R3
```

2. (3 puntos) En un sistema con procesador ARM y una memoria de 64KB se quiere analizar el comportamiento del siguiente fragmento de un programa.

```
.global start
.equ N, 12

.data
A: .word N valores separados por comas
B: .word N valores separados por comas

.bss
C: .space 4*N

.text
....
mov R3, #N
mov R4, #0
mov R5, #0
mov R2, #0
for:
cmp R2, R3
bge fin
ldr R4, [R0, R2, LSL#2]
ldr R5, [R1, R2, LSL#2]
add R4, R4, R5
str R4, [R7, R2, LSL#2]
add R2, R2, #1
b for
fin:
```

1. Asumiendo que el programa se enlaza ubicando la sección .data a partir de la **dirección 0x0090** y la sección .bss a continuación de la sección .data, y que el fragmento de la sección .text en la que se encuentra la primera instrucción (mov R3, #N) se ubica en la **dirección 0x0140**, y que R0, R1 y R7 contienen las direcciones de A, B y C respectivamente; responde de forma razonada a las siguientes preguntas:

- (0.5 puntos) Obtener el rango de direcciones que ocupa el fragmento de programa y cada uno de los arrays. ¿Cuántos bloques de memoria principal ocupan? ¿Qué bloques son?
- (1.5 puntos) El sistema dispone de caches separadas de datos (D\$) e instrucciones (I\$) de 32 bytes cada una, ambas con política de emplazamiento directo, y políticas de escritura directa (write through) con asignación en escritura (write allocate), y tamaño de bloque de 16 bytes. Obtener los fallos que se producirán en la cache de datos (D\$) y en la cache de instrucciones (I\$) al ejecutar el fragmento del programa. Indicar, además del número total de fallos, para qué bloques de memoria principal se producen los fallos.
- (1 punto) Ahora añadimos al sistema de memoria una cache de segundo nivel unificada para datos e instrucciones (L2\$) de 128 bytes, asociativa por conjuntos de 2 vías, con reemplazamiento LRU y políticas de escritura directa (write through) con asignación en escritura (write allocate). Indicar el número de fallos que se producen en L2 tras ejecutar el fragmento del programa. (El tamaño de bloque es también de 16 bytes).

Solución:

- (0.5 puntos) Obtener el rango de direcciones que ocupa el programa con la tabla de literales, y cada uno de los arrays ¿Cuántos bloques de memoria principal ocupan? ¿Qué bloques son?

Programa = 12 instrucciones: Ocupan tres Bloques de MP: B_{MP} 20 , B_{MP}21, B_{MP}22,

Rango de direcciones: 0x0140 - 0x 017F

Array A: Ocupan dos Bloques de MP: B_{MP} 9, B_{MP}10 y B_{MP} 11 Rango de direcciones: 0x0090 - 0x 00BF

Array B: Ocupan dos Bloques de MP: B_{MP} 12, B_{MP} 13 y B_{MP}14, Rango de direcciones: 0x00C0 - 0x 00EF

Array C: Ocupan dos Bloques de MP: B_{MP} 15, B_{MP} 16 y B_{MP}17, Rango de direcciones: 0x00F0 - 0x 011F

b) (1.5 puntos)

11 1 4

| Etiqueta | Bc | Posicion |

Si no tenemos en cuenta la segmentación

Ciclo	Cache	Bloque Mem	Bloque Cache
Fetch mov R3, #N	I\$	20	0
Fetch cmp R2, R3	I\$	21	1
Mem ldr R4, [R0, R2, LSL#2]	D\$	9	1
Mem ldr R5, [R1, R2, LSL#2]	D\$	12	0
Fetch add R4,R4,R5	I\$	22	0 (reemplazo B20)
Mem str R4, [R7, R2, LSL#2]	D\$	15	1 (reemplazo B9)
Mem ldr R4, [R0, R2, LSL#2]	D\$	9	1 (reemplazo B15)
Mem str R4, [R7, R2, LSL#2]	D\$	15	1 (reemplazo B9)
...			

Fallos en la I\$: 3

Fallos en la D\$: 2 por iteración (A y C) + 1 cada 4 iteraciones (B) = 24 + 3 = 27.

c) (1 puntos) Muestra la evolución del contenido de L2 (etiqueta y contenido de cada bloque) tras ejecutar el programa. ¿Cuántos fallos a L2 se producen tras ejecutar el programa completo?

12		4
B _{MP}		Byte del bloque
10	2	4
Etiqueta	Conj	Byte del bloque
11	1	4
Etiqueta	B _C	Byte del bloque

Ciclo	Bloque Mem	Conjunto Cache
Fetch mov R3, #N	20	0
Fetch cmp R2, R3	21	1

Mem ldr	R4, [R0, R2, LSL#2]	9	1
Mem ldr	R5, [R1, R2, LSL#2]	12	0
Fetch add	R4, R4, R5	22	2
Mem str	R4, [R7, R2, LSL#2]	15	3
Mem ldr	R4, [R0, R2, LSL#2]	9	2
Mem str	R4, [R7, R2, LSL#2]	15	3
...			

Como vemos, el bloque que contiene las instrucciones de la primera mitad del bucle va al mismo conjunto de cache que el bloque que contiene el primer bloque de A. Pero como hay 2 vías no hay reemplazo.

La correspondencia de bloques con conjunto de cache es la siguiente:

Bloque	Conjunto L2
9	1
10	2
11	3
12	0
13	1
14	2
15	3
20	0
21	1
22	2

Como vemos, hay algunos conflictos, pero nunca hay más de dos activos por conjunto. Por lo tanto, tendremos sólo los fallos de inicio en la cache (1 por bloque accedido): 12 fallos en total.

3.- En una empresa dedicada al análisis de datos tienen que utilizar un algoritmo que requiere sumar todos los elementos de cada columna de una matriz de 512x512 elementos. El código C de dicho algoritmo es el siguiente:

```
for (j = 0; j < 512; j++)
    for (i = 0; i < 512; i++)
        red[j] = red[j] + A[i][j];
```

Sabiendo que:

- En C los elementos de una matriz se emplazan en memoria consecutivos por filas
- El procesador es de 32 bits (direcciones virtuales de 32 bits).
- El array de enteros A[512][512] se ubica en el mapa virtual de memoria del proceso a partir de la dirección **0x00458000**, y el array de enteros red[512] se ubica justo a continuación de A.
- El sistema operativo (SO) maneja páginas de 4KB con reemplazo **LRU**.

se pide:

- (0.75 puntos) Indicar qué páginas de memoria virtual ocupan los arrays A y red, indicando qué elementos de cada array están en cada página.
- (1.25 puntos) El SO le asigna al proceso 8 marcos de página (lo que sería equivalente a decir que, desde el punto de vista de este proceso, la memoria física tiene 8 páginas). Sabiendo que 2 de esos 8 marcos están

ocupados por la página de código y la página de pila, accedidos constantemente y que por tanto nunca son elegidos para el reemplazo, y que los 6 restantes no contienen inicialmente ninguna página de A ni de red; indicar razonadamente cuántos fallos de página se producirán al ejecutar el código y el número de accesos a A y red que no suponen fallo de página.

- c) **(1 punto)** Razone si es posible o no reducir significativamente los fallos de página del proceso con alguna modificación en su código, indicando en su caso qué transformación se haría y cuantificando la mejora esperada.

Solución

a)

El array A está alineado con el tamaño de página.

El array red también, puesto que el tamaño de A es múltiplo del tamaño de página.

Cada fila de los arrays ocupa $512 \times 4B = 2KB$, media página, por tanto hay dos filas por página.

El array A ocupa por tanto 256 (0x100) páginas, de la 0x00458 a la 0x00557 ambas incluidas.

El array red ocupa la primera mitad de la página 0x00558.

En la página i [0-255] del array A (pág 0x00458 + i) están los elementos de las filas $2*i$ y $2*i + 1$.

En la única página de red están todos los elementos de red.

b)

Los 6 marcos van siendo ocupados por las páginas de datos. En la primera iteración, al referenciar $A[0][0]$ se produce un fallo de página y se traen las dos primeras filas de A. También se produce fallo al referenciar el elemento $red[0]$, y se trae la página que ocupa el array red. En la siguiente iteración no hay fallo ($A[1][0]$). En la tercera iteración se produce fallo al referenciar $A[2][0]$, pero no hay fallo al referenciar red.

Por tanto, al procesar la primera columna se producen $512/2 + 1$ fallos de página y 3×512 accesos a memoria. Se han traído 257 páginas, que han ido reemplazando a las últimas referenciadas usando los 6 marcos disponibles. Como la página usada para red se referencia en cada iteración, nunca es seleccionada para reemplazo.

Al procesar la siguiente columna, la página de red está cargada en un marco, pero las primeras páginas de A, ocupadas por las primeras filas, ya no están. Por tanto, se produce un fallo de página cada dos iteraciones. Esto se repite para cada columna.

Fallos de página: $512/2 \times 512 + 1 = 131073$.

Numero total de accesos a A y red: $512 \times 512 \times 3 = 786432$.

Número de accesos a A y red que no suponen fallo: $512 \times 512 \times 3 - (512/2 \times 512 + 1) = 655359$

c) El algoritmo permite un intercambio de bucles:

```
for (i = 0; i < 512; i++)
    for (j = 0; j < 512; j++)
        red[j] = red[j] + A[i][j];
```

En este caso, al comienzo de cada fila par se produce un fallo de página, se trae la página con las dos filas, se recorren todos sus elementos y ya no se vuelven a acceder. El número de fallos que se producirían sería de $512/2=256$, es decir, los fallos se reducen unas 512 veces.



ESTRUCTURA DE COMPUTADORES
Examen Convocatoria Ordinaria. 3 de Junio de 2019

Nombre _____ DNI _____
Apellidos _____ Grupo _____

1. **(4 puntos)** Sea un procesador segmentado de cinco etapas con las siguientes características:

- Un dato se puede leer y escribir en el banco de registros en el mismo ciclo de reloj.
- Existe anticipación de operandos. El cortocircuito se realiza siempre desde los registros del pipeline, y existe cortocircuito a la etapa ID para las instrucciones de salto, que se resuelven en dicha etapa.
- La detección de riesgos LDE y generación de paradas se realiza en la etapa de decodificación.
- Los riesgos estructurales referidos a memoria se detectan y se resuelven mediante espera en decodificación.
- Los riesgos de EDE entre dos instrucciones A y B tal que A precede a B se resuelven mediante inhibición de escritura de la instrucción A.
- No se implementan saltos retardados, sino que se busca siempre la siguiente instrucción al salto y se cancela su ejecución si el salto es tomado.
- Las unidades funcionales de las que dispone el procesador son las siguientes:

UF	Cantidad	Latencia	Segmentación
FP ADD	1	2	Sí
FP MUL	1	4	Sí
INT ALU	1	1	No

El siguiente código calcula el producto escalar de dos vectores, realizando dos productos por iteración:

```
MOV R1, #64
bucle: LS F0, 0(R1)
      LS F2, 128(R1)
      MULS F4, F0, F2
      ADDS F6, F6, F4
      SUBI R1, R1, #4
      LS F0, 0(R1)
      LS F2, 128(R1)
      MULS F4, F0, F2
      ADDS F6, F6, F4
      SUBI R1, R1, #4
      BNEZ R1, bucle
      SS F6, 256(R1)
```

- (1.5 puntos)** Represente el diagrama instrucción-tiempo para la primera iteración, indicando los cortocircuitos realizados, las paradas y sus causas.
- (1 punto)** Proponga una optimización del código que disminuya el número de ciclos de ejecución. Realice una estimación del ahorro conseguido.
- (1.5 puntos)** Considere ahora la ejecución del código de la siguiente tabla. Indique y justifique el valor de las señales StallF, FlushE, A1, RsE, RdE, ForwardBE y ForwardAE en los ciclos 5, 6, 7 y 8.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
mov r1, #64	IF	ID	E	M	WB									
bucle: lw r2, 0(r1)		IF	ID	E	M	WB								
lw r3, 128(r1)			IF	ID	E	M	WB							
mul r4, r3, r2				IF	IDp	ID	E	M	WB					
add r6, r6, r4					IFp	IF	ID	E	M	WB				
subi r1, r1, #4							IF	ID	E	M	WB			
bnez r1, bucle								IF	IDp	ID	E	M	WB	
sw r6, 256(r1)									IFp	IF	ID	E	M	WB

2. **(3 puntos)** Un pequeño microcontrolador basado en ARM dispone de una memoria principal de 64KB con cachés separadas de datos e instrucciones de 64B cada una, de emplazamiento directo y bloques de 16B. La caché de datos tiene asignación en escritura (write allocate) y actualización en post-escritura (write-back). En dicho sistema se quiere evaluar el rendimiento del siguiente fragmento de un programa:

```
for (i=0; i < 40; i++)
    A[i] += B[i];
```

que al compilarse resulta en el siguiente código ensamblador, ubicado en la dirección 0xC020:

```
    ldr r5, =A
    ldr r6, =B
    mov r4, #40
L1:  ldr r0, [r5]
    ldr r1, [r6], #4
    add r0, r1, r0
    str r0, [r5], #4
    subs r4, r4, #1
    bgt L1
```

Sabiendo que A y B son arrays de 40 enteros (un entero son 4 bytes), que están ubicados en memoria uno a continuación del otro a partir de la dirección 0x1000, y que el contenido de los directorios al comienzo de la ejecución de este fragmento de código es el siguiente:

IL1		
	v	etiqueta
0	1	0x301
1	1	0x301
2	1	0x300
3	1	0x300

DL1		
	v	etiqueta
0	1	0x042
1	1	0x042
2	1	0x044
3	1	0x044

Responda razonadamente a las siguientes cuestiones:

- (1.5 puntos)** ¿Cuál es la tasa de fallos de ambas cachés al ejecutar el código?
- (0.75 puntos)** Una modificación algorítmica requiere que los arrays pasen a ser de 48 enteros. Indique cómo afectaría esto al rendimiento del código. Indique la nueva tasa de fallos de ambas cachés.
- (0.75 puntos)** Proponga alguna transformación de código que mejore el rendimiento de la caché con arrays de 48 enteros, indicando la tasa de fallos que obtiene en ese caso.

3. **(3 puntos)** En una máquina con una memoria DRAM de 256MB y un procesador de 32 bits (direcciones virtuales de 32 bits) el sistema operativo gestiona la memoria virtual por paginación, utilizando páginas de 4KB. En dicha máquina se ejecuta un servidor que ofrece consultas a una base de datos. Para ser más rápido, el servidor mantiene mapeados en su espacio virtual de memoria los 8 ficheros de la base de datos consultados más recientemente. Cada fichero ocupa 1KB. En un momento dado los ficheros mapeados en memoria virtual son los indicados por la tabla de la izquierda. La tabla de la derecha nos muestra las entradas más relevantes de la tabla de páginas del proceso.

Ubicación de ficheros en M.V.

Fichero	D. Virtual
Fch 1	0x000A0000
Fch 3	0x00020000
Fch 2	0x000A0C00
Fch 4	0x00020400
Fch 8	0x000A0800
Fch 12	0x00060C00
Fch 13	0x000A0400
Fch 20	0x00060800

Tabla de Páginas

v	P. Física
0	...
1	0x0021
0	...
0	0x0003
1	0x0040
0	...
1	0x0001
0	...

Se pide:

- (1 punto)** Indique para cada fichero ubicado en memoria virtual si se encuentra en memoria DRAM (física) o no, y en caso afirmativo el rango de direcciones físicas que ocupa.
- (1 punto)** El sistema dispone además de una caché física de 4 bloques de 64 bytes por bloque, de emplazamiento directo y una TLB de dos entradas completamente asociativa. En un momento dado los contenidos del directorio de la caché y la TLB son los siguientes (LRU = 0 es el que se ha accedido más recientemente):

Directorio

	v	Etiqueta
0	1	0x00014
1	1	0x00014
2	1	0x00215
3	1	0x00215

TLB

P. Virtual	P. Física	LRU
0x000A0	0x0001	0
0x00020	0x0021	1

Indique cuál es el fichero que ha sido accedido más recientemente, justificando la respuesta.

- (1 punto)** Si a continuación el proceso intenta acceder al segundo bloque que forma parte de Fch 12, indique las estructuras que se consultarían, el orden en que se haría y si habría fallo o acierto en cada caso. En caso de que se produjeran fallos indique cómo quedaría cada una de las estructuras tras resolver el fallo. En caso de fallo de página asumir que se usa como página física libre la 0x0002.

Examen EC – Convocatoria ordinaria 2019

Ejercicio 2

Un pequeño microcontrolador basado en ARM dispone de una memoria principal de 64KB con caches separadas de datos e instrucciones de 64B cada una, de emplazamiento directo y bloques de 16B. La cache de datos tiene asignación en escritura (write allocate) y actualización en post-escritura (write-back). En dicho sistema se quiere evaluar el rendimiento del siguiente fragmento de un programa:

```
for (i=0; i < 40; i++)  
    A[i] += B[i];
```

que al compilarse resulta en el siguiente código ensamblador, cuya primera instrucción se ubica en la dirección 0xC020.

```
...  
ldr r5, =A  
ldr r6, =B  
mov r4, #40  
L1:  ldr r0, [r5]  
     ldr r1, [r6], #4  
     add r0, r1, r0  
     str r0, [r5], #4  
     subs r4, r4, #1  
     bgt L1
```

Sabiendo que A y B son arrays de 40 enteros, que están ubicados en memoria uno a continuación del otro a partir de la dirección 0x1000, y que el contenido de los directorios es el siguiente:

IL1		
	v	etiqueta
0	1	0x301
1	1	0x301
2	1	0x300
3	1	0x300

DL1		
	v	etiqueta
0	1	0x042
1	1	0x042
2	1	0x044
3	1	0x044

Responda razonadamente a las siguientes cuestiones:

- (1.5 puntos)** Cuál es la tasa de fallos de ambas caches al ejecutar el código.
- (0.5 puntos)** Se propone la siguiente modificación de código para mejorar el rendimiento:

```
for (i=39; i >=0 ; i--)  
    A[i] += B[i];
```

Explica el motivo e indica cuál sería la nueva tasa de fallos de ambas cachés (asume que el código ensamblador generado ocupa los mismos bloques que antes).

- c) **(0.5 puntos)** Una modificación algorítmica requiere que los arrays pasen a ser de 48 enteros. Indica cómo afectaría esto al rendimiento del código original. Indica la nueva tasa de fallos de ambas cachés.
- d) **(0.5 puntos)** Propón alguna transformación de código que mejore el rendimiento de la caché con arrays de 48 enteros, indicando la tasa de fallos que obtienes en ese caso.

SOLUCIÓN

a) Formato de dirección: Etiqueta | M | P
 10 2 4

El código ocupa tres bloques (0xC02, 0xC03, 0xC04), que en la caché estarían en los bloques 2 y 3 con etiqueta 0x300 y en el bloque 0 con etiqueta 0x301. Como esos bloques están en la IL1 no hay fallos en la cache de instrucciones, su tasa de fallos para este código es el 0%.

Ambos arrays tienen sus direcciones de comienzo alineadas con el tamaño de bloque. A ocupa 10 bloques (0x100 hasta el 0x109), con etiquetas 0x040 a 0x042. B ocupa otros 10 bloques (0x10A hasta el 0x113), con etiquetas 0x42 a 0x044. En la caché están los dos últimos bloques de A y de B.

Elementos	M	Etiqueta	Elementos	M	Etiqueta
A[0-3]	0	0x040	B[0-3]	2	0x042
A[4-7]	1	0x040	B[4-7]	3	0x042
A[8-11]	2	0x040	B[8-11]	0	0x043
A[12-15]	3	0x040	B[12-15]	1	0x043
A[16-19]	0	0x041	B[16-19]	2	0x043
A[20-23]	1	0x041	B[20-23]	3	0x043
A[24-27]	2	0x041	B[24-27]	0	0x044
A[28-31]	3	0x041	B[28-31]	1	0x044
A[32-35]	0	0x042	B[32-35]	2	0x044
A[36-39]	1	0x042	B[36-39]	3	0x044

Al ejecutar el código original los dos primeros bloques de A y B reemplazarán a los bloques finales que están en la cache. Como los bloques de A y de B no compiten por el mismo marco de cache, no hay fallos por conflicto. Tendremos un fallo por cada bloque accedido. En el str no hay fallo puesto que el bloque se ha traído por el ldr inicial. Por lo tanto, tendremos dos fallos cada 4 iteraciones, dando un total de $40/4 * 2 = 20$ fallos. El número de accesos es de 3 por iteración, 120 en total. La tasa de fallos será por tanto de $20 / 120 = 0.166.. \sim 17\%$.

b) Como el código ocupa los mismos bloques que antes, la tasa de fallos seguirá siendo del 0% en la IL1.

El nuevo código aprovecha el contenido de la cache en ese punto del programa, empezando a procesar los últimos bloques, para los que no habrá fallo. Los fallos totales pasarán de 20 a 16 y la tasa de fallos en la cache DL1 se reduce al $16/120 = 0.133... \sim 13\%$.

c) Si los arrays pasan a ser de 48 enteros, los elementos $A[i]$ y $B[i]$ estarán en bloques de memoria que compiten por el mismo bloque de cache. Esto hará que en la primera iteración se produzcan 3 fallos (A, B y A) y en las siguientes dos fallos (B y A), dando un total de $3+2*40 = 83$ fallos. La tasa de fallos pasará a ser de $83 / 120 = 0.69.. \sim 69\%$.

d) Para evitar los conflictos entre A y B podemos hacer fusión de arrays o array padding en A. En ambos casos eliminaríamos el conflicto entre A y B y pasaríamos a tener sólo un fallo por bloque, quedándonos en total 24 fallos. La tasa de fallos volvería a ser la misma que en el apartado a: $24 / (3 * 48) = 0.166.. \sim 17\%$

Ejercicio 3

En una máquina con una memoria DRAM de 256MB y un procesador de 32 bits (direcciones virtuales de 32 bits) el sistema operativo gestiona la memoria virtual por paginación, utilizando páginas de 4KB. En dicha máquina ejecuta un servidor que ofrece consultas a una base de datos. Para ser más rápido, el servidor mantiene en memoria una copia de los 8 ficheros de la base de datos consultados más recientemente. Cada fichero ocupa 1KB. En un momento dado el programa tiene mapeados en memoria los ficheros indicados por la tabla de la izquierda. La tabla de la derecha nos muestra las entradas más relevantes de la tabla de páginas del proceso.

Ubicación de ficheros en M.V.

Fichero	D. Virtual
Fch 1	0x000A0000
Fch 3	0x00020000
Fch 2	0x000A0C00
Fch 4	0x00020400
Fch 8	0x000A0800
Fch 12	0x00060C00
Fch 13	0x000A0400
Fch 20	0x00060800

Tabla de Páginas

	v	P. Física
...	0	...
0x00020	1	0x0021
...	0	...
0x00060	0	0x0003
0x00061	1	0x0040
...	0	...
0x000A0	1	0x0001
...	0	...

Se pide:

- (1 punto)** Indica para cada fichero ubicado en memoria virtual si se encuentra en memoria DRAM (física) o no, y en caso afirmativo el rango de direcciones físicas que ocupa.
- (1 punto)** El sistema dispone además de una cache física de 4 bloques de 64 bytes por bloque, de emplazamiento directo y una TLB de dos entradas completamente asociativa. En un

momento dado los contenidos del directorio de la cache y la TLB son los siguientes (LRU = 0 es el que se ha accedido más recientemente):

Directorio

	v	Etiqueta
0	1	0x00014
1	1	0x00014
2	1	0x00215
3	1	0x00215

TLB

P. Virtual	P. Física	LRU
0x000A0	0x0001	0
0x00020	0x0021	1

Indica cuál es el fichero que ha sido accedido más recientemente, justificando la respuesta.

- c) **(1 punto)** Si a continuación el proceso intenta acceder al segundo bloque que forma parte de Fich 12, indica las estructuras que se consultarían, el orden en que se haría, si hay fallo o acierto en cada caso. En caso de que se produjesen fallos indica como quedaría cada una de las estructuras tras resolver el fallo. En caso de fallo de página asumir que se usa como página física libre la 0x0002.

SOLUCION:

a)

Fichero	D. Virtual	En DRAM	Rango D. Físicas
Fch 1	0x000A0000	sí	0x0001000 - 0x00013FF
Fch 3	0x00020000	sí	0x0021000 - 0x00213FF
Fch 2	0x000A0C00	sí	0x0001C00 - 0x0001FFF
Fch4	0x00020400	sí	0x0021400 - 0x00217FF
Fch 8	0x000A0800	sí	0x0001800 - 0x0001BFF
Fch 12	0x00060C00	no	
Fch 13	0x000A0400	sí	0x0001400 - 0x00017FF
Fch 20	0x00060800	no	

b) A partir de la cache podemos obtener los rangos virtuales accedidos más recientemente:

	Etiqueta	Rango físico	Rango virtual	fichero
0	0x00014	0x0001400-0x000143F	0x000A0400-0x000A043F	Fich 13
1	0x00014	0x0001440-0x000147F	0x000A0440-0x000A047F	Fich 13
2	0x00215	0x0021580-0x00215BF	0x00020580-0x000205BF	Fich 4
3	0x00215	0x00215C0-0x00215FF	0x000205C0-0x000205FF	Fich 4

La información LRU de la TLB nos dice cuál es la traducción realizada más recientemente, concretamente la página 0x000A0, por lo que el fichero accedido más recientemente es el que corresponde a esta página, Fich 13.

c) La dirección virtual de comienzo de este bloque sería: 0x00060C40. Primero se accedería a la TLB para traducir la página virtual 0x00060, dando lugar a un fallo de TLB. La MMU o en su defecto el SO accederían a la tabla de páginas, al comprobar que no está la página presente en memoria se produciría un fallo de página. Entraría el sistema operativo, que buscaría la página en disco y la asignaría la página física 0x0002. La TLB quedaría actualizada del siguiente modo:

TLB

P. Virtual	P. Física	LRU
0x000A0	0x0001	1
0x00060	0x0002	0

La dirección física resultante sería la 0x0002C40, se accedería a la cache en el bloque 1 con la etiqueta 0x0002C. Sería un fallo de cache, el controlador de cache traería el nuevo bloque quedando el directorio como sigue:

	v	Etiqueta
0	1	0x00014
1	1	0x0002C
2	1	0x00215
3	1	0x00215



ESTRUCTURA DE COMPUTADORES
Examen Convocatoria Extraordinaria. Junio de 2019

Nombre _____ DNI _____
Apellidos _____ Grupo _____

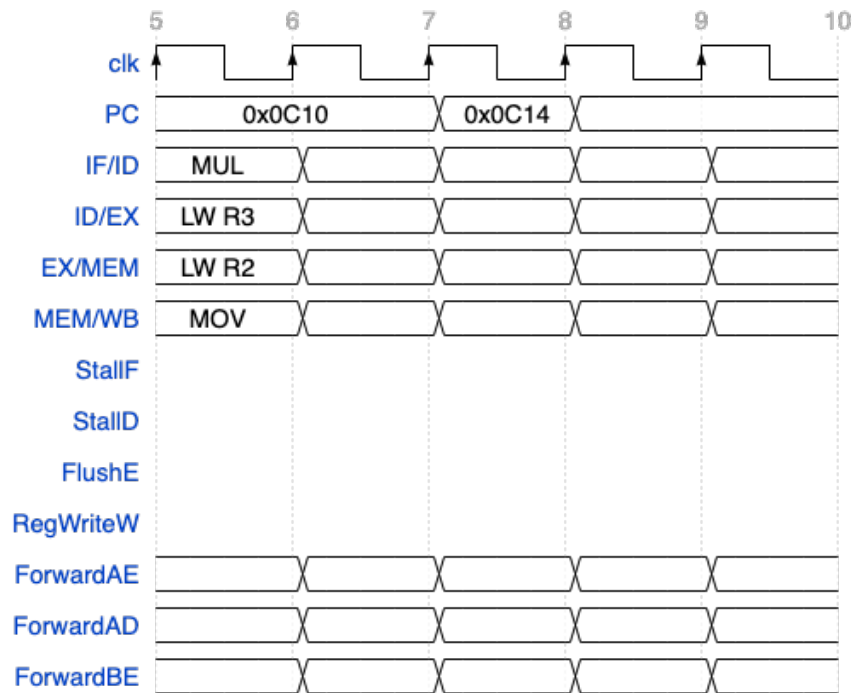
1. (4 puntos) Sea un procesador segmentado de cinco etapas con las siguientes características:

- Un dato se puede leer y escribir en el banco de registros en el mismo ciclo de reloj.
- Existe anticipación de operandos. El cortocircuito se realiza siempre desde los registros del pipeline, y existe cortocircuito a la etapa ID para las instrucciones de salto, que se resuelven en dicha etapa.
- La detección de riesgos LDE y generación de paradas se realiza en la etapa de decodificación.
- Los riesgos estructurales referidos a memoria se detectan y se resuelven mediante espera en decodificación.
- Los riesgos de EDE entre dos instrucciones A y B tal que A precede a B se resuelven mediante inhibición de escritura de la instrucción A.
- No se implementan saltos retardados, sino que se busca siempre la siguiente instrucción al salto y se cancela su ejecución si el salto es tomado.
- Las unidades funcionales de las que dispone el procesador son las siguientes:

UF	Cantidad	Latencia	Segmentación
FP ADD	1	2	Sí
FP MUL	1	3	Sí
INT ALU	1	1	No

a. (1 punto) Dado el siguiente diagrama instrucción-tiempo, complete el cronograma. Supóngase que la unidad de detección de riesgos es combinacional. Indíquese también sobre el cronograma los flancos en los que se produce la escritura del banco de registros.

Addr.	Ciclo	1	2	3	4	5	6	7	8	9	10	11	12	13
0x0C04	bucle: LW R2, 0(R1)		IF	ID	E	M	WB							
0x0C08	LW R3, 128(R1)			IF	ID	E	M	WB						
0x0C0C	MUL R4, R3, R2				IF	IDp	ID	E	M	WB				
0x0C10	ADD R6, R6, R4					IFp	IF	ID	E	M	WB			
0x0C14	SUBI R1, R1, #4							IF	ID	E	M	WB		
0x0C18	BNEZ R1, bucle								IF	IDp	ID	E	M	WB
0x0C1C	SW R6, 256(R1)									IFp	IF	ID	---	---



- b. (2 puntos) Represente el diagrama instrucción-tiempo para la primera iteración del código que se muestra a continuación, indicando los cortocircuitos realizados, las paradas y sus causas. Inicialmente R1=64 y R2=1.

```

MOV    R2, #1
loop:  BEQ    R0, R2, mult
      LW     F0, 0(R1)
      LW     F1, 128(R1)
      MULS   F4, F0, F1
      ADDS   F5, F5, F4
      MOV    R2, #0
mult:  LW     F0, 0(R1)
      ADDS   F5, F5, F0
      SUBI   R1, R1, #4
      BNEZ   R0, R1, loop
      SW     F5, 256(R1)
      MOV    R2, #1

```

- c. (1 punto) Aumentar la frecuencia de reloj del procesador de 1 GHz a 1.5 GHz impone usar una caché de datos segmentada lo que requiere segmentar las etapas de memoria y de decodificación en dos etapas cada una de ellas, incrementando el total de etapas del procesador hasta 7. En este caso, la detección de riesgos LDE, la generación de paradas y el cortocircuito a la etapa ID para las instrucciones de salto se resuelven en la segunda etapa de decodificación. Determinar el speedup de la ejecución de todas las iteraciones del código en esta implementación tomando la implementación inicial (analizada en el apartado b.) como caso base.

2. **(3 puntos)** Un pequeño microcontrolador basado en ARM dispone de una memoria principal de 512KB con cachés separadas de datos e instrucciones de 128B cada una, de emplazamiento directo y bloques de 16B. La caché de datos tiene asignación en escritura (write allocate). En dicho sistema se quiere evaluar el rendimiento del siguiente fragmento de un programa:

```
for (i=0; i < 6; i++)  
    for (j=1; j<7; j++)  
        A[i][j] = (B[i][j]+ B[i][j+1])/2;
```

Sabiendo que A y B son matrices de 6x8 de enteros (un entero son 4 bytes), que están ubicadas en memoria en las direcciones B: 0x15000 y A: 0x20310 y que se almacenan por filas. Responda razonadamente a las siguientes cuestiones:

- a. **(1.5 puntos)** ¿Cuál es la tasa de fallos de la caché de datos al ejecutar el código?
- b. **(1.5 puntos)** Si se añade al microcontrolador una cache de víctimas de 32B totalmente asociativa y con algoritmo de reemplazamiento LRU repetir el apartado a, detallando qué bloques se encuentran en la caché de datos y en la de víctimas al finalizar la primera iteración del bucle externo (i ==0).
3. **(3 puntos)** En una máquina con una memoria DRAM de 256MB y un procesador de 32 bits (direcciones virtuales de 32 bits) el sistema operativo gestiona la memoria virtual por paginación, utilizando páginas de 4KB y con algoritmo de reemplazamiento LRU. En dicha máquina se ejecuta una aplicación que accede a distintas estructuras de datos, cada una de las cuales ocupa 1KB. El proceso dispone de los marcos de página 6, 7, 8 y 9 para almacenar el código y los datos (se asignan en ese orden, esto es, la primera página que necesite el proceso se alojará en el marco 6). Se dispone de una TLB de dos entradas totalmente asociativa.

Las direcciones virtuales iniciales de cada una de las estructuras de datos son:

Nombre E. D.	A	B	C	D	E	F
D. V. inicial	0x00128000	0x00941600	0x00002000	0x005110A0	0x00300100	0x00041000

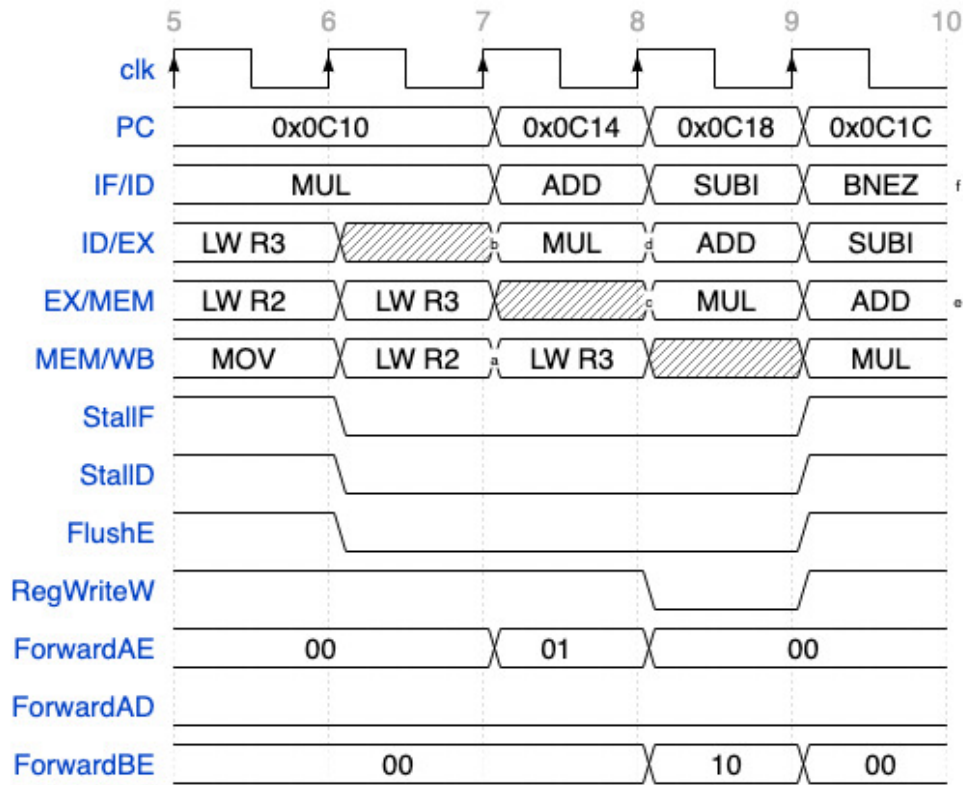
El código empieza en la dirección virtual 0x000C0000 y ocupa 3500B. Tener en cuenta que cada vez que se ejecuta una instrucción se accede a la página de código que almacena esta instrucción.

Se pide:

- a. **(1.5 puntos)** Mostrar el estado final de la tabla de páginas y la TLB cuando la aplicación se ejecuta y accede a las estructuras de datos en el siguiente orden: A, B, A, C, D, A, E, B, A.
- b. **(1.0 puntos)** Si a continuación el proceso intenta acceder a F, indique las estructuras que se consultarían, el orden en que se haría y si habría fallo o acierto en cada caso. En caso de que se produjeran fallos indique cómo quedaría cada una de las estructuras tras resolver el fallo.
- c. **(0.5 puntos)** Si la memoria cache de datos fuera virtualmente accedida físicamente marcada, ¿cuál sería su tamaño máximo para asociatividad 2?

Examen EC 2019 Convocatoria extraordinaria

Ejercicio 1 a)



Ejercicio 1 c)

El periodo de reloj para la implementación b) es 1ns, el número de ciclos es:

$$\text{num_ciclos} = 20 + 10 \cdot 15 + 5 = 175$$

$$t_{\text{ejecucion}} = 175 \text{ ns}$$

Para la implementación c) el periodo de reloj es 0,667 ns,

$$\text{num_ciclos} = 23 + 13 \cdot 15 + 7 = 235$$

$$t_{\text{ejec}} = 156,745 \text{ ns.}$$

$$\text{Speedup} = 1,116$$

Ejercicio 1.b (1ª iter)	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
MOV R2, #1	IF	ID	E	M	W																				
loop:BEQ R0, R2, mult		IF	IDp	ID	E	M	W																		
LW F0, 0(R1)			IFp	IF	ID	E	M	W																	
LW F1, 128(R1)					IF	ID	E	M	W																
MULS F4, F0, F1						IF	IDp	ID	Mul1	Mul2	Mul3	M	W												
ADDS F5, F5, F4							IFp	IF	IDp	IDp	ID	Add1	Add2	M	W										
MOV R2, #0									IFp	IFp	IF	IDp	ID	E	M	W									
mult: LW F0, 0(R1)												IFp	IF	ID	E	M	W								
ADDS F5, F5, F0														IF	IDp	ID	Add1	Add2	M	W					
SUBI R1, R1, #4															IFp	IF	IDp	ID	E	M	W				
BNEZ R0, R1, loop																	IFp	IF	IDp	ID	E	M	W		
SW F5, 256(R1)																			IFp	IF	----	----	----	----	
loop:BEQ R0, R2, mult																					IF	ID	E	M	W

Ejercicio 1.b (2ª iter)	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
loop:BEQ R0, R2, mult	IF	ID	E	M	W										
LW F0, 0(R1)		IF	----	----	----	----									
LW F1, 128(R1)															
MULS F4, F0, F1															
ADD5 F5, F5, F4															
MOV R2, #0															
mult: LW F0, 0(R1)			IF	ID	E	M	W								
ADD5 F5, F5, F0				IF	IDp	ID	Add1	Add2	M	W					
SUBI R1, R1, #4					IFp	IF	IDp	ID	E	M	W				
BNEZ R0, R1, loop							IFp	IF	IDp	ID	E	M	W		
SW F5, 256(R1)									IFp	IF	----	----	----	----	
MOV R2, #1	Última iteración										IF	ID	E	M	W

Ejercicio 1.c (1ª iter)	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
MOV R2, #1	IF	ID1	ID2	E	M1	M2	W																		
loop:BEQ R0, R2, mult		IF	ID1	ID2p	ID2	E	M1	M2	W																
LW F0, 0(R1)			IF	ID1p	ID1	ID2	E	M1	M2	W															
LW F1, 128(R1)				IFp	IF	ID1	ID2	E	M1	M2	W														
MULS F4, F0, F1						IF	ID1	ID2p	ID2p	ID2	Mul1	Mul2	Mul3	M1	M2	W									
ADDS F5, F5, F4							IF	ID1p	ID1p	ID1	ID2p	ID2p	ID2	Add1	Add2	M1	M2	W							
MOV R2, #0								IFp	IFp	IF	ID1p	ID1p	ID1	ID2p	ID2	E	M1	M2	W						
mult: LW F0, 0(R1)											IFp	IFp	IF	ID1p	ID1	ID2	E	M1	M2	W					
ADDS F5, F5, F0														IFp	IF	ID1	ID2p	ID2p	ID2	Add1	Add2	M1	M2	W	
SUBI R1, R1, #4																IF	ID1p	ID1p	ID1	ID2p	ID2	E	M1	M2	W
BNEZ R0, R1, loop																	IFp	IFp	IF	ID1p	ID1	ID2p	ID2	E	M1
SW F5, 256(R1)																				IFp	IF	ID1p	ID1	----	----
loop:BEQ R0, R2, mult																								IF	ID1

Ejercicio 1.c (2ª iter)	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
loop: BEQ R0, R2, mult	IF	ID1	ID2	E	M1	M2	W													
LW F0, 0(R1)		IF	ID1	----	----	----	----	----												
LW F1, 128(R1)																				
MULS F4, F0, F1																				
ADDS F5, F5, F4																				
MOV R2, #0																				
mult: LW F0, 0(R1)				IF	ID1	ID2	E	M1	M2	W										
ADDS F5, F5, F0					IF	ID1	ID2p	ID2p	ID2	Add1	Add2	M1	M2	W						
SUBI R1, R1, #4						IF	ID1p	ID1p	ID1	ID2p	ID2	E	M1	M2	W					
BNEZ R0, R1, loop							IFp	IFp	IF	ID1p	ID1	ID2p	ID2	E	M1	M2	W			
SW F5, 256(R1)										IFp	IF	ID1p	ID1	----	----					
loop: BEQ R0, R2, mult														IF	ID1	ID2	E	M1	M2	W
MOV R2, #1	última iteración													IF	ID1	ID2	E	M1	M2	W

Examen EC – Convocatoria extraordinaria 2019

Ejercicio 2

(3 puntos) Un pequeño microcontrolador basado en ARM dispone de una memoria principal de 512KB con cachés separadas de datos e instrucciones de 128B cada una, de emplazamiento directo y bloques de 16B. La caché de datos tiene asignación en escritura (write allocate). En dicho sistema se quiere evaluar el rendimiento del siguiente fragmento de un programa:

```
for (i=0; i < 6; i++)
    for (j=1; j<7; j++)
        A[i][j] = (B[i][j] + B[i][j+1])/2;
```

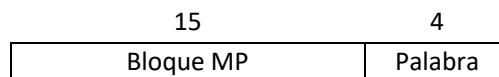
Sabiendo que A y B son matrices de 6x8 de enteros (un entero son 4 bytes), que están ubicadas en memoria en las direcciones B: 0x15000 y A: 0x20310 y que se almacenan por filas. Responda razonadamente a las siguientes cuestiones:

- (1.5 puntos) ¿Cuál es la tasa de fallos de la caché de datos al ejecutar el código?
- (1.5 puntos) Si se añade al microcontrolador una cache de víctimas de 32B totalmente asociativa y con algoritmo de reemplazamiento LRU repetir el apartado a, detallando qué bloques se encuentran en la caché de datos y en la de víctimas al finalizar la primera iteración del bucle externo (i == 0).

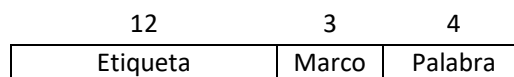
Formato de la instrucción:

Direcciones de 19 bits

MP:



MC:



- a) Para calcular la tasa de fallos necesitamos en nº de fallos de MC y el nº total de referencias.

En cada bloque hay 4 elementos de la matriz.

Cada matriz tiene 6x8= 48 elementos, comienza en la palabra 0 de un bloque de MP y ocupa 48/4=12 bloques completos y contiguos de la MP.

Matriz B: 0x15000 → Bloques de MP: 0x1500 – 0x150B → Se ubican en los marcos: 0 – 7 y 0 – 3.

Matriz A: 0x20310 → Bloques de MP: 0x2031 – 0x203C → Se ubican en los marcos: 1 – 7 y 0 – 4.

1ª iteración: B[0][1], B[0][2], A[0][1] → Fallo B[0][1], acierto B[0][2] y fallo A[0][1]

Fallo B[0][1] → MC: Marco 0 (B[0][0]-B[0][3]),...

Fallo A[0][1] → MC: Marco 0 (B[0][0]-B[0][3]), Marco 1 (A[0][0]-A[0][3]), ...

2ª iteración: B[0][2], B[0][3], A[0][2] → Acierto B[0][2], acierto B[0][3] y acierto A[0][2]

3ª iteración: B[0][3], B[0][4], A[0][3] → Acierto B[0][3], fallo B[0][4] y fallo A[0][3]

Fallo B[0][4] → MC: Marco 0 (B[0][0]-B[0][3]), Marco 1 (B[0][4]-A[0][7]), ...

Fallo A[0][3] → MC: Marco 0 (B[0][0]-B[0][3]), Marco 1 (A[0][0]-A[0][3]), ...

4ª iteración: B[0][4], B[0][5], A[0][4] → Fallo B[0][4], acierto B[0][5] y fallo A[0][4]

Fallo B[0][4] → MC: Marco 0 (B[0][0]-B[0][3]), Marco 1 (B[0][4]-A[0][7]), ...

Fallo A[0][4] →

MC: Marco 0 (B[0][0]-B[0][3]), Marco 1 (B[0][4]-A[0][7]), Marco 2 (A[0][4]-A[0][7]), ...

5ª iteración: B[0][5], B[0][6], A[0][5] → Acierto B[0][5], acierto B[0][6] y acierto A[0][5]

6ª iteración: B[0][6], B[0][7], A[0][6] → Acierto B[0][6], acierto B[0][7] y acierto A[0][6]

Esto se repite (con otros elementos de las matrices y otros marcos de MC) en cada iteración del bucle externo.

Nº de fallos/ iteración del bucle externo: 6

Nº fallos totales: $6 \times 6 = 36$

Nº accesos a memoria: $6 \times 6 \times 3 = 108$

Tasa de fallos = $\text{Nº Fallos} / \text{Nº Accesos} = 36 / 108 = 0,33$

b) Cache víctima de 32 bytes (2 bloques)

1ª iteración: B[0][1], B[0][2], A[0][1] → Fallo B[0][1], acierto B[0][2] y fallo A[0][1]

MC: Marco 0 (B[0][0]-B[0][3]), Marco 1 (A[0][0]-A[0][3]), ...

Cache víctima: vacía

2ª iteración: B[0][2], B[0][3], A[0][2] → Acierto B[0][2], acierto B[0][3] y acierto A[0][2]

3ª iteración: B[0][3], B[0][4], A[0][3]

B[0][4] fallo: MC: Marco 0 (B[0][0]-B[0][3]), Marco 1 (B[0][4]-B[0][7]), ...

Cache víctima: Bloque A[0][0]-A[0][3], ...

A[0][3] fallo de cache y acierto cache víctima:

MC: Marco 0 (B[0][0]-B[0][3]), Marco 1 (A[0][0]-A[0][3]), ...

Cache víctima: Bloque B[0][4]-B[0][7], ...

4ª iteración: B[0][4], B[0][5], A[0][4] → Fallo B[0][4], acierto B[0][5] y fallo A[0][4]

B[0][4] fallo de cache y acierto cache víctima:

MC: Marco 0 (B[0][0]-B[0][3]), Marco 1 (B[0][4]-B[0][7]), ...

Cache víctima: Bloque A[0][0]-A[0][3], ...

A[0][4] fallo:

MC: Marco 0 (B[0][0]-B[0][3]), Marco 1 (B[0][4]-B[0][7]), Marco 2 (A[0][4]-A[0][7])...

Cache víctima: Bloque A[0][0]-A[0][3], ...

5ª iteración: B[0][5], B[0][6], A[0][5] → Acierto B[0][5], acierto B[0][6] y acierto A[0][5]

6ª iteración: B[0][6], B[0][7], A[0][6] → Acierto B[0][6], acierto B[0][7] y acierto A[0][6]

Nº de fallos/ iteración del bucle externo: 6

Nº fallos totales: $6 \cdot 4 = 24$

Nº accesos a memoria: $6 \cdot 6 \cdot 3 = 108$

Tasa de fallos = $\text{Nº Fallos} / \text{Nº Accesos} = 24 / 108 = 0,22$

Ejercicio 3

(3 puntos) En una máquina con una memoria DRAM de 256MB y un procesador de 32 bits (direcciones virtuales de 32 bits) el sistema operativo gestiona la memoria virtual por paginación, utilizando páginas de 4KB y con algoritmo de reemplazamiento LRU. En dicha máquina se ejecuta una aplicación que accede a distintas estructuras de datos, cada una de las cuales ocupa 1KB. El proceso dispone de los marcos de página 6, 7, 8 y 9 para almacenar el código y los datos (se asignan en ese orden, esto es, la primera página que necesite el proceso se alojará en el marco 6). Se dispone de una TLB de dos entradas totalmente asociativa.

Las direcciones virtuales iniciales de cada una de las estructuras de datos son:

Nombre E. D.	A	B	C	D	E	F
D. V. inicial	0x00128000	0x00941600	0x00002000	0x005110A0	0x00300100	0x00041000

El código empieza en la dirección virtual 0x000C0000 y ocupa 3500B. Tener en cuenta que cada vez que se ejecuta una instrucción se accede a la página de código que almacena esta instrucción.

Se pide:

- (1.5 puntos) Mostrar el estado final de la tabla de páginas y la TLB cuando la aplicación se ejecuta y accede a las estructuras de datos en el siguiente orden: A, B, A, C, D, A, E, B, A.
- (1.0 puntos) Si a continuación el proceso intenta acceder a F, indique las estructuras que se consultarían, el orden en que se haría y si habría fallo o acierto en cada caso. En caso de que se produjeran fallos indique cómo quedaría cada una de las estructuras tras resolver el fallo.
- (0.5 puntos) Si la memoria cache de datos fuera virtualmente accedida físicamente marcada, ¿cuál sería su tamaño máximo para asociatividad 2?

Formato de la dirección:

MV: 32 bits

20	12
Nº Página Virtual	Desplazamiento

MP: DRAM 256 MB

16	12
Nº Página Física	Desplazamiento

a) A, B, A, C, D, A, E, B, A

Cada estructura ocupa 1 KB: (sumar 0x3FF a la dirección de comienzo de la E.D. para calcular su rango de direcciones).

Estructura de datos A: 0x00128000 - 0x001283FF → Página virtual 0x00128

Estructura de datos B: 0x00941600 - 0x009419FF → Página virtual 0x00941

Estructura de datos C: 0x00002000 - 0x000023FF → Página virtual 0x00002

Estructura de datos D: 0x005110A0 - 0x0051149F → Página virtual 0x00511

Estructura de datos E: 0x00300100 - 0x003004FF → Página virtual 0x00300

Estructura de datos F: 0x00041000 - 0x000413FF → Página virtual 0x00041

El código comienza en 0x000C0000 y ocupa 3500B → Cada página ocupa 2^{12} bytes = 4096 bytes y el código comienza en la palabra 0 de la página por lo que ocupa una sola página

Código: 0x000C0000 → Página virtual 0x000C0

El código se está continuamente referenciando, por lo que podemos dejar fijo su LRU a 0. También es la primera página que se lleva a la MP y por tanto al marco 6.

Referencias: Código, A, B, A, C, D, A, E, B, A

Tabla de páginas											
Nº Pág. Virtual	Válido / LRU										Nº Pág. Física
0x00002 (C)					1/1	1/2	1/3	0/3	0/3	0/3	9
0x00041 (F)										0/-	-
0x000C0 (Código)*	1/0	1/0	1/0	1/0	1/0	1/0	1/0	1/0	1/0	1/0	6
0x00128 (A)		1/1	1/2	1/1	1/2	1/3	1/1	1/2	1/3	1/1	7
0x00300 (E)								1/1	1/2	1/3	9
0x00511 (D)						1/1	1/2	1/3	0/3	0/3	8
0x00941 (B)			1/1	1/2	1/3	0/3	0/3	0/3	1/1	1/2	8
Referencias	*	A	B	A	C	D	A	E	B	A	

TLB	
Nº Pág. Virtual	Nº Pág. Física
0x000C0	6
0x00128	7

b) Referencia F

1º Se accede a la TLB buscando la página virtual 0x00041 y no se encuentra

2º Se accede a la tabla de páginas y se comprueba que el bit de válido es 0, por lo que la página buscada no está en la MP

3º El sistema operativo trae a MP la página buscada. Con el algoritmo LRU la ubica en la página física 9 reemplazando a la página 0x00300 que tenía la estructura de datos E.

4º Se actualizan los datos LRU en la tabla de páginas

5º Se añade la entrada correspondiente a la página virtual 0x00041 en la TLB

Tabla de páginas			
Nº Pág. Virtual	Válido	LRU	Nº Pág. Física
0x00002 (C)	0	3	9
0x00041 (F)	1	0	9
0x000C0 (Código)	1	1	6
0x00128 (A)	1	2	7
0x00300 (E)	0	3	9
0x00511 (D)	0	3	8
0x00941 (B)	1	3	8

TLB	
Nº Pág. Virtual	Nº Pág. Física
0x000C0	6
0x00041	9

c) Memoria cache virtualmente accedida físicamente marcada de asociatividad 2.

El campo desplazamiento de la dirección virtual son 12 bits, por tanto el tamaño máximo de la cache tendría 12 bits para los campos conjunto y palabra (o byte).

Con asociatividad de grado 2 el tamaño máximo de la cache sería: $2 \cdot 2^{12} = 8 \text{ KB}$

	ESTRUCTURA DE COMPUTADORES Examen - 11 de junio de 2021
	Nombre _____ DNI _____

1. (1 punto)

- (0,5 pts) Expresar en punto flotante de precisión simple, según el estándar IEEE 754, el número $(10,25)_{10}$.
- (0,5 pts) Expresar en decimal el número $0xBE60_0000$, representado en punto flotante de precisión simple según el estándar IEEE 754.

2. (3 pts) Sea un procesador segmentado de cinco etapas con las siguientes características:

- Un dato se puede leer y escribir en el banco de registros en el mismo ciclo de reloj.
- Existe anticipación de operandos. El cortocircuito se realiza siempre desde los registros del pipeline, y existe cortocircuito a la etapa ID para las instrucciones de salto, que se resuelven en dicha etapa.
- La detección de riesgos LDE y generación de paradas se realiza en la etapa de decodificación.
- Los riesgos estructurales referidos a memoria se detectan y se resuelven mediante espera en la última etapa de ejecución.
- Los riesgos de EDE entre dos instrucciones A y B tal que A precede a B se resuelven mediante inhibición de escritura de la instrucción A.
- No se implementan saltos retardados, sino que se busca siempre la siguiente instrucción al salto y se cancela su ejecución si el salto es tomado.
- Las unidades funcionales de las que dispone el procesador son las siguientes:

UF	Cantidad	Latencia	Segmentación
FP ADD	1	4	Sí
FP MUL	1	5	Sí
INT ALU	1	1	No

- (2 pts) Completar el diagrama instrucción-tiempo para los primeros 24 ciclos del siguiente fragmento de código, indicando claramente los cortocircuitos realizados, las paradas y sus causas.

```

loop:  ADDD    F6, F5, F2
        ADDD    F2, F1, F3
        SUBI    R5, R5, #4
        LD      F4, 0(R5)
        ADDD    F6, F6, F2
        MUL    F5, F6, F4
        BNE    R5, R0, loop
        SD      F5, 256(R5)

```

- (1 pto) Calcular el CPI de la ejecución completa del código, siendo $R0=0$ y $R5=32$ al comienzo de la ejecución.

3. (3 pts) En un computador con caches separadas de datos e instrucciones, de 128 bytes cada una, emplazamiento directo y bloques de 16 bytes, se quiere ejecutar el siguiente código C:

```

#define N 64;
int A[N][N];    /* Un entero ocupa 32 bits
int B[N];
int C[N];

for (i=0; i < N; i++)
    for (j=0; j < N; j++)
        temp=temp + B[j] * A[i][j];
    C[i] =temp;

```

Ignorando los accesos a la variable **temp** y suponiendo que los tres arrays se colocan en memoria de modo consecutivo a partir de la dirección $0x0B00_0000$.

- a) (1 pto) Calcular el número de fallos de bloque en la cache de datos si se usa escritura directa sin asignación en escritura
- b) (1 pto) Si añadimos una memoria cache de segundo nivel unificada con bloques de 32 bytes, asociativa por conjuntos con dos vías y tamaño 256 bytes. Calcular el número de fallos de bloque por datos en cada nivel de cache si en ambos niveles de cache se usa escritura directa con asignación en escritura.
- c) (1 pto) Si la latencia de acceso a la cache L2 es 10 ciclos y la tasa de transferencia entre L1 y L2 es de 16 bytes por ciclo, y la latencia de acceso a memoria principal desde L2 es de 50 ciclos y la tasa de transferencia es de 8 bytes por ciclo, calcular la penalización que se ha producido por los fallos de acceso a memoria cache. Suponer que los tiempos de escritura se solapan con los de lectura.
4. (3 ptos) En una máquina con una memoria principal de 4 KB el sistema operativo gestiona la memoria virtual por paginación, utilizando páginas de 1KB y un algoritmo LRU de reemplazo de páginas. Siendo el tamaño de la memoria virtual 8 KB, se pide:
- a) (1 pto) Indica cuál es el contenido de la tabla de páginas y de una TLB de 2 entradas (LRU), tras el acceso a las siguientes direcciones virtuales de memoria: 0x04C0, 0x1680, 0x09C4, 0x06A0 y 0x0CCC, suponiendo que anteriormente la memoria principal estaba vacía y que las páginas físicas se han asignado en orden creciente a partir de la página 0. Utiliza el valor LRU = 0 para la última página referenciada.
- b) (1 pto) En un momento dado el contenido de la tabla de páginas es el que se muestra a continuación. Indica cómo evoluciona la tabla de páginas tras realizar las siguientes referencias a memoria: 0x12FC, 0x0540, 0x18C0, 0x0380, 0x1C44.
Si a continuación se referencia de nuevo la dirección 0x12FC, indica cuáles son sus direcciones físicas cada vez que se referenció. ¿Son iguales? Razona la respuesta.

	Válido	PF	LRU
0	0		
1	1	0	2
2	0		
3	1	3	0
4	1	2	3
5	0		
6	0		
7	1	1	1

- c) (0,5 ptos) Se añade una memoria cache de 128 bytes con bloques de 32 bytes, virtualmente accedida, completamente asociativa y con algoritmo de reemplazo FIFO. En un momento dado el contenido de la TLB y de la memoria cache es el que se muestra a continuación. Mostrar los rangos, que se puedan con la información disponible, de direcciones físicas y virtuales de los bloques ubicados en la memoria cache.

Bloque MC	Etiqueta	FIFO
0	0x45	2
1	0x8A	0
2	0xFF	1
3	0x68	3

TLB		
PV	PF	LRU
2	2	0
4	3	1

- d) (0,5 ptos) Partiendo del contenido de la memoria cache y de la TLB del apartado anterior, indica detalladamente y en orden, qué sucede cuando se referencia la dirección de memoria 0x1280.
El valor FIFO = 0 indica el último bloque que se ha traído a la memoria cache.

Ejercicios punto flotante

Expresar en punto flotante de precisión simple, según el estándar IEEE 754, el número $(10,25)_{10}$.

Expresar en decimal el número 0xBE600000, representado en punto flotante de precisión simple según el estándar IEEE 754.

Soluciones

$(10,25)_{10}$

Convertimos la parte entera a base 2: $(10)_{10} = (1010)_2$

Convertimos la parte fraccionaria a base 2: $(.25)_{10} = (.01)_2$

Sumamos ambos valores: $1010 + 0.01 = 1010.01$

En binario normalizado: 1.01001×2^3

Añadimos el sesgo 127 al exponente 3: $127 + 3 = 130 \rightarrow$ en binario: 1000 0010

Escribimos el número en el orden (signo - exponente - mantisa)

0 1000 0010 0100 1000 0000 0000 0000 000 en hexadecimal $\rightarrow$ **0x41240000**

0xBE600000

Escribimos el número en binario y separamos (signo - exponente - mantisa)

1 - 01111100 - 110 0000 0000 0000 0000 0000

Signo: -

Exponente: $2^6 + 2^5 + 2^4 + 2^3 + 2^2 - 127 = 124 - 127 = -3$

El nº es: $-1.11 \times 2^{-3} = -(2^0 + 2^{-1} + 2^{-2}) \times 2^{-3} = -(1 + 0,5 + 0,25) \times 0,125 = -1,75 \times 0,125 = -0,21875$

Ejercicio Segmentación

Ejercicio 2 (3 pts). Sea un procesador segmentado de cinco etapas con las siguientes características:

- Un dato se puede leer y escribir en el banco de registros en el mismo ciclo de reloj.
- Existe anticipación de operandos. El cortocircuito se realiza siempre desde los registros del pipeline, y existe cortocircuito a la etapa ID para las instrucciones de salto, que se resuelven en dicha etapa.
- La detección de riesgos LDE y generación de paradas se realiza en la etapa de decodificación.
- Los riesgos estructurales referidos a memoria se detectan y se resuelven mediante espera en la última etapa de ejecución.
- Los riesgos de EDE entre dos instrucciones A y B tal que A precede a B se resuelven mediante inhibición de escritura de la instrucción A.
- No se implementan saltos retardados, sino que se busca siempre la siguiente instrucción al salto y se cancela su ejecución si el salto es tomado.
- Las unidades funcionales de las que dispone el procesador son las siguientes:

UF	Cantidad	Latencia	Segmentación
FP ADD	1	4	Sí
FP MUL	1	5	Sí
INT ALU	1	1	No

- a. (2 pts) Completar el diagrama instrucción-tiempo para los primeros 24 ciclos del siguiente fragmento de código, indicando claramente los cortocircuitos realizados, las paradas y sus causas.

```
loop:  ADDD  F6, F5, F2
      ADDD  F2, F1, F3
      SUBI  R5, R5, #4
      LD    F4, 0(R5)
      ADDD  F6, F6, F2
      MULD  F5, F6, F4
      BNE   R5, R0, loop
      SD    F5, 256(R5)
```

- b. (1 pts) Calcular el CPI de la ejecución completa del código, siendo R0=0 y R5=32 al comienzo de la ejecución.

			1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35
loop:	ADDD	F6, F5, F2	IF	ID	A1	A2	A3	A5	M	WB																											
	ADDD	F2, F1, F3		IF	ID	A1	A2	A3	A4	M	WB																										
	SUBI	R5, R5, #4			IF	ID	EX	M	WB																												
	LD	F4, 0(R5)				IF	ID	EXp	EXp	EX	M	WB																									
	ADDD	F6, F6, F2					IF	IDp	IDp	ID	A1	A2	A3	A4	M	WB																					
	MULD	F5, F6, F4						IFp	IFp	IF	IDp	IDp	IDp	ID	M1	M2	M3	M4	M5	M	WB																
	BNE	R5, R0, loop									IFp	IFp	IFp	IF	ID	EX	M	WB																			
	SD	F5, 256(R5)													IF	X	X	X	X																		
loop:	ADDD	F6, F5, F2														IF	IDp	IDp	ID	A1	A2	A3	A4	M	WB												
	ADDD	F2, F1, F3																IFp	IFp	IF	ID	A1	A2	A3	A4	M	WB										
	SUBI	R5, R5, #4																		IF	ID	EX	M	WB													
	LD	F4, 0(R5)																			IF	ID	EXp	EXp	EX	M	WB										
	ADDD	F6, F6, F2																				IF	IDp	IDp	ID	A1	A2	A3	A4	M	WB						
	MULD	F5, F6, F4																					IFp	IFp	IF	IDp	IDp	IDp	ID	M1	M2	M3	M4	M5	M	WB	
	BNE	R5, R0, loop																							IFp	IFp	IFp	IF	ID	EX	M	WB					
	SD	F5, 256(R5)																											IF	IDp	IDp	IDp	ID	EX	M	WB	

$$CPI = (13+15*7+7)/(7*8+1)=2,193$$

1. En un computador con caches separadas de datos e instrucciones, de 128 bytes cada una, emplazamiento directo y bloques de 16 bytes, se quiere ejecutar el siguiente código C:

```
#define N 64
int A[N][N];      /* Un entero ocupa 32 bits
int B[N];
int C[N]

for (i=0; i < N; i++)
    for (j=0; j < N; j++)
        temp=temp + B[j] * A[i][j]
C[i] =temp
```

Ignorando los accesos a la variable **temp** y suponiendo que los tres arrays se colocan en memoria de modo consecutivo a partir de la dirección 0x0B00_0000.

- Calcular el número de fallos de bloque si se usa escritura directa sin asignación en escritura
- Si añadimos una memoria cache de segundo nivel con bloques de 32 bytes, asociativa por conjuntos con dos vías y tamaño 256 bytes. Calcular el número de fallos de bloque si en ambos niveles de cache se usa escritura directa con asignación en escritura.
- Si la latencia de acceso a la cache L2 es 10 ciclos y la tasa de transferencia entre L1 y L2 es de 16 bytes por ciclo, y la latencia de acceso a memoria principal desde L2 es de 50 ciclos y la tasa de transferencia es de 8 bytes por ciclo, calcular la penalización que se ha producido por los fallos de acceso a memoria cache

Soluciones

Etiqueta	Marco	Byte
25	3	4

- a) Bloques de 16 bytes → 4 elementos de A, B ó C por bloque

A ocupa $64 \times 64 / 4 = 1024$ bloques

B y C ocupan $64 / 4 = 16$ bloques

A → 0x0B000000 (Marco 0)

Las filas de A empiezan en 0x0B000000, 0x0B000100, 0x0B000200, 0x0B000300... Todas estas direcciones compiten por el mismo marco (Marco 0).

B → $0x0B000000 + 0x4000 = 0x0B004000$ (Marco 0)

C → $0x0B004000 + 0x100 = 0x0B004100$ (Marco 0)

En todas las iteraciones A y B compiten por los mismos marcos. Todas las referencias a A y B producen fallo.

Fallos lectura = $64 \times 64 \times 2 = 8192$

Fallos de escritura (escrituras en MP) = 64

- b) M. Cache L1

Etiqueta	Marco	Byte
25	3	4

M. Cache L2

Etiqueta	Conjunto	Byte
25	2	5

Ejercicio Memoria Virtual

En una máquina con una memoria principal de 4 KB el sistema operativo gestiona la memoria virtual por paginación, utilizando páginas de 1KB y un algoritmo LRU de reemplazo de páginas. Siendo el tamaño de la memoria virtual 8 KB, se pide:

- a) Indica cuál es el contenido de la tabla de páginas invertida y de una TLB de 2 entradas, tras el acceso a las siguientes direcciones de memoria: 0x04C0, 0x1680, 0x09C4, 0x06A0 y 0x0CCC, suponiendo que anteriormente la memoria principal estaba vacía y que las páginas físicas se han asignado en orden creciente. Utiliza el valor LRU = 0 para la última página referenciada.

- b) En un momento dado el contenido de la tabla de páginas es el que se muestra a continuación. Indica cómo evoluciona la tabla de páginas tras realizar las siguientes referencias a memoria: 0x12FC, 0x0540, 0x18C0, 0x0380, 0x1C44. Si a continuación se referencia de nuevo la dirección 0x12FC, indica cuáles son sus direcciones físicas cada vez que se referenció. ¿Son iguales? Razona la respuesta.

	Válido	PF	LRU
0	0		
1	1	0	2
2	0		
3	1	3	0
4	1	2	3
5	0		
6	0		
7	1	1	1

- c) Se añade una memoria cache de 128 bytes con bloques de 32 bytes, virtualmente accedida, completamente asociativa y con algoritmo de reemplazo FIFO. En un momento dado el contenido de la TLB y de la memoria cache es el que se muestra a continuación. Indica los rangos de direcciones físicas y virtuales de los bloques ubicados en la memoria cache.

Bloque MC	Etiqueta	FIFO
0	0x45	2
1	0x8A	0
2	0xFF	1
3	0x68	3

TLB:

PV	PF	LRU
2	2	0
4	3	1

- d) Partiendo del contenido de la memoria cache y de la TLB del apartado anterior, indica detalladamente y en orden, qué sucede cuando se referencia la dirección de memoria 0x1280. El valor FIFO = 0 indica el último bloque que se ha traído a la memoria cache.

Soluciones

a) Formatos de la dirección virtual y física:

NPV	Desplazamiento
3	10

NPF	Desplazamiento
2	10

0x04C0: Página virtual 001 (1)

0x1680: Página virtual 101 (5)

0x09C4: Página virtual 010 (2)

0x06A0: Página virtual 001 (1)

0x0CCC: Página virtual 011 (3)

Tabla de páginas invertida:

PV	PF	LRU
1	0	1
5	1	3
2	2	2
3	3	0

TLB:

PV	PF	LRU
1	0	1
3	3	0

b) 0x12FC, 0x0540, 0x18C0, 0x0380, 0x1C44 y 0x12FC

0x12FC: PV 4

0x0540: PV 1

0x18C0: PV 6

0x0380: PV 0

0x1C44: PV 7

	V	PF	LRU	V	PF	LRU	V	PF	LRU	V	PF	LRU	V	PF	LRU	V	PF	LRU
0	0			0			0			1	3	0	1	3	1	1	3	2
1	1	0	3	1	0	0	1	0	1	1	0	2	1	0	3	0	0	3
2	0			0			0			0			0			0		
3	1	3	1	1	3	2	1	3	3	0	3	3	0	3	3	0	3	3
4	1	2	0	1	2	1	1	2	2	1	2	3	0	2	3	1	0	0
5	0			0			0			0			0			0		
6	0			0			1	1	0	1	1	1	1	1	2	1	1	3
7	1	1	2	1	1	3	0	1	3	0	1	3	1	2	0	1	2	1
	0x12FC			0x0540			0x18C0			0x0380			0x1C44			0x12FC		

0x12FC: 100 1011111100

1ª referencia: 101011111100 – Dirección física: 0xAFC

2ª referencia: 001011111100 – Dirección física: 0x2FC

c) Formato dirección para la MC:

Etiqueta	Byte
8	5

Bloque 0:

Direcciones virtuales: 0100 0101 00000 - 0100 0101 11111 → 0x08A0 – 0x08BF

Direcciones físicas: 100 0101 00000 - 100 0101 11111 → 0x8A0 – 0x8BF

Bloque 1:

Direcciones virtuales: 1000 1010 00000 – 1000 1010 11111 → 0x1140 – 0x115F

Direcciones físicas: 110 1010 00000 – 110 1010 11111 → 0xD40 – 0xD5F

Bloque 2:

Direcciones virtuales: 1111 1111 00000 – 1111 1111 11111 → 0x1FE0 – 0x1FFF

Direcciones físicas: No tenemos información para obtenerlas porque la PV 7 no está en la TLB y no se da la tabla de páginas

Bloque 3:

Direcciones virtuales: 0110 1000 00000 – 0110 1000 11111 → 0x0D00 – 0x0D1F

Direcciones físicas: No tenemos información para obtenerlas porque la PV 3 no está en la TLB y no se da la tabla de páginas

d) 0x1280

Se accede a la cache con la dirección virtual

1 0010 1000 0000 → Etiqueta: 1 0010 100 (0x94)

Fallo de Cache → Traer bloque de MP → Traducir la dirección usando TLB → Se actualiza LRU en TLB

1 0010 1000 0000 → Pág. Virtual 4 → Pág. Física 3 → Dirección física:

111010000000 (0xE80) → Se copia el bloque 0x74 de MP a la MC que quedaría:

Bloque MC	Etiqueta	FIFO
0	0x45	3
1	0x8A	1
2	0xFF	2
3	0x94	0

TLB:

PV	PF	LRU
2	2	1
4	3	0

En L2:

A: 512 bloques $\rightarrow 512/64 = 8$ bloques por fila de A

B: 8 bloques

En el bucle interno tendríamos 1 fallo en cache L2 por cada bloque accedido de A y B.

Fallos bucle interno: $8 (A) + 8 (B) = 16$

En el bucle externo tendríamos:

Nº total fallos de lectura en L2: $64 * 16 = 1024$

Nº total fallos de escritura en L2: 64

Total fallos en L2= 1088

En L1 los mismos fallos que en a)

Nº total fallos de lectura en L1: $64 * 64 * 2 = 8192$

Nº total fallos de escritura en L1 (los bloques se traen a la MC): 64

Total fallos en L1= 8256

c) Penalización = $\text{FallosL1} * (\text{LatenciaL2} + \text{BloqueL2aL1}) + \text{FallosL2} * (\text{LatenciaMP} + \text{BloqueMPaL2})$

Penalización = $8256 * (10 + 1) + 1088 * (50 + 4) = 90816 + 58752 = 149568$ ciclos